TUGAS PENDAHULUAN / TUGAS UNGUIDED
PEMROGRAMAN PERANGKAT BERGERAK

MODUL 01

[JUDUL MODUL 01]



Disusun Oleh :

Kevin kendra adi sebastian /

103122400067 SE-08-02

Asisten Praktikum :

Abu Abdirrahman Humaid Al-Atsary
Aflah Rizkyadhafin Nurfikri

Dosen Pengampu :

Yudha Islami Sulistya, S.Kom., M.Cs.

PROGRAM STUDI S1 SOFTWARE ENGINEERING FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

JURNAL MODUL 01

A. SOAL

JURNAL PPB MODUL 1

TUGAS:

1. **Membuat Project Flutter**
   Buatlah sebuah project Flutter baru dengan nama **MOD1_NIM** menggunakan **Visual Studio Code**. Dokumentasikan / *screenshot* setiap langkah pengerjaan mulai dari membuat *project* hingga berhasil menjalankan *program*-nya.

2. **Pastikan** project **dapat dijalankan** pada emulator Android.

KETERANGAN PENGUMPULAN:

- Kumpulkan **file PDF** yang berisi *screenshot* tampilan langkah pengerjaan beserta dengan hasilnya, sertakan juga **code**-nya.
- Kumpulkan **file ZIP** yang berisi folder **lib** dari **project** yang telah dibuat.

B. JAWABAN

Source code

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        // This is the theme of your application.
        //
        // TRY THIS: Try running your application with "flutter run". You'll see
        // the application has a purple toolbar. Then, without quitting the app,
        // try changing the seedColor in the colorScheme below to Colors.green
        // and then invoke "hot reload" (save your changes or press the "hot
```

```
        // reload" button in a Flutter-supported IDE, or press
"r" if you used
        // the command line to start the app).
        //
        // Notice that the counter didn't reset back to zero;
the application
        // state is not lost during the reload. To reset the
state, use hot
        // restart instead.
        //
        // This works for code too, not just values: Most code
changes can be
        // tested with just a hot reload.
        colorScheme: .fromSeed(seedColor: Colors.deepPurple),
      ),
      home: const MyHomePage(title: 'Flutter Demo Home Page'),
    );
  }
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});

  // This widget is the home page of your application. It is
stateful, meaning
  // that it has a State object (defined below) that contains
fields that affect
  // how it looks.

  // This class is the configuration for the state. It holds the
values (in this
  // case the title) provided by the parent (in this case the
App widget) and
  // used by the build method of the State. Fields in a Widget
subclass are
  // always marked "final".

  final String title;

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0;
```

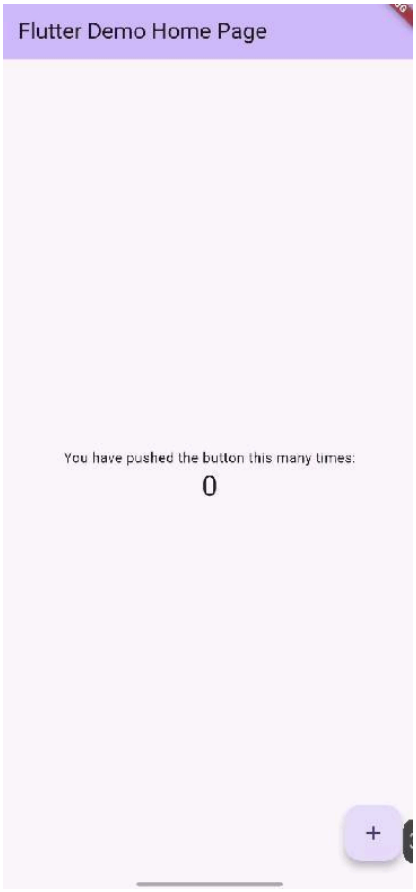
```
  void _incrementCounter() {
    setState(() {
      // This call to setState tells the Flutter framework that
something has
      // changed in this State, which causes it to rerun the
build method below
      // so that the display can reflect the updated values. If
we changed
      // _counter without calling setState(), then the build
method would not be
      // called again, and so nothing would appear to happen.
      _counter++;
    });
  }

  @override
  Widget build(BuildContext context) {
    // This method is rerun every time setState is called, for
instance as done
    // by the _incrementCounter method above.
    //
    // The Flutter framework has been optimized to make
rerunning build methods
    // fast, so that you can just rebuild anything that needs
updating rather
    // than having to individually change instances of widgets.
    return Scaffold(
      appBar: AppBar(
        // TRY THIS: Try changing the color here to a specific
color (to
        // Colors.amber, perhaps?) and trigger a hot reload to
see the AppBar
        // change color while the other colors stay the same.
        backgroundColor:
Theme.of(context).colorScheme.inversePrimary,
        // Here we take the value from the MyHomePage object
that was created by
        // the App.build method, and use it to set our appbar
title.
        title: Text(widget.title),
      ),
      body: Center(
        // Center is a layout widget. It takes a single child
and positions it
```

```
        // in the middle of the parent.
        child: Column(
          // Column is also a layout widget. It takes a list of
children and
          // arranges them vertically. By default, it sizes
itself to fit its
          // children horizontally, and tries to be as tall as
its parent.
          //
          // Column has various properties to control how it
sizes itself and
          // how it positions its children. Here we use
mainAxisAlignment to
          // center the children vertically; the main axis here
is the vertical
          // axis because Columns are vertical (the cross axis
would be                          // horizontal).
                                   //
                                   // TRY THIS: Invoke "debug
                                   painting" (choose the
"Toggle Debug Paint"
          // action in the IDE, or press "p" in the console), to
see the                           // wireframe
                                   for each
                                   widget.
                                   mainAxisAlignm
                                   ent: .center,
times:'),                          children: [
                                     const Text('You have pushed the
                                     button this many

                                     Text(
                                       '$_counter',
                                       style:
                                       Theme.of(context).textTheme.h
                                       adlineMedium,
                                     ),
                                   ],
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _incrementCounter,
        tooltip: 'Increment',
        child: const Icon(Icons.add),
      ),
```

Screenshot Output



Deskripsi Program

Program ini merupakan aplikasi Flutter sederhana berbasis antarmuka grafis (GUI) yang berfungsi sebagai penambah angka (counter) interaktif. Aplikasi ini tidak menggunakan algoritma pemrosesan yang kompleks, melainkan mengandalkan algoritma manajemen status (state management) serta penanganan acara (event handling) bawaan kerangka kerja Flutter. Cara kerja program dimulai ketika aplikasi dijalankan melalui fungsi main(), yang membangun tampilan awal layar dengan judul "Flutter Demo Home Page" serta menampilkan nilai awal pembanding _counter bernilai 0. Setiap kali pengguna menekan tombol berikon tambah (+) di pojok kanan bawah, tombol tersebut memicu fungsi _incrementCounter() yang menjalankan perintah setState(). Fungsi setState() tersebut secara otomatis memperbarui nilai variabel _counter dengan menambahkan nilainya sebesar 1, sekaligus menginstruksikan sistem untuk membangun ulang tampilan teks pada layar (rebuild widget). Hasil atau output yang dihasilkan oleh program ini adalah sebuah antarmuka aplikasi dengan bilah judul (AppBar) berwarna ungu, tulisan

"You have pushed the button this many times:", dan angka di bawahnya yang nilainya bertambah secara real-time setiap kali tombol tambah ditekan.